```
    return done;
}
```

This is the recommended way to wait for condition variables with a time limit, if you're not passing a predicate to the wait. This way, the overall length of the loop is bounded. As you saw in section 4.1.1, you need to loop when using condition variables if you don't pass in the predicate, in order to handle spurious wakeups. If you use `wait_for()` in a loop, you might end up waiting almost the full length of time before a spurious wake, and the next time through the wait time starts again. This may repeat any number of times, making the total wait time unbounded.

With the basics of specifying timeouts under your belt, let's look at the functions that you can use the timeout with.

4.3.4. Functions that accept timeouts

The simplest use for a timeout is to add a delay to the processing of a particular thread, so that it doesn't take processing time away from other threads when it has nothing to do. You saw an example of this in section 4.1, where you polled a "done" flag in a loop. The two functions that handle this are `std::this_thread::sleep_for()` and `std::this_thread::sleep_until()`. They work like a basic alarm clock: the thread goes to sleep either for the specified duration (with `sleep_for()`) or until the specified point in time (with `sleep_until()`). `sleep_for()` makes sense for examples like that from section 4.1, where something must be done periodically, and the elapsed time is what matters. On the other hand, `sleep_until()` allows you to schedule the thread to wake at a particular point in time. This could be used to trigger the backups at midnight, or the payroll print run at 6:00 a.m., or to suspend the thread until the next frame refresh when doing a video playback.

Of course, sleeping isn't the only facility that takes a timeout; you already saw that you can use timeouts with condition variables and futures. You can even use timeouts when trying to acquire a lock on a mutex if the mutex supports it. Plain `std::mutex` and `std::recursive_mutex` don't support timeouts on locking, but `std::timed_mutex` does, as does `std::recursive_timed_mutex`. Both these types support `try_lock_for()` and `try_lock_until()` member functions that try to obtain the lock within a specified time period or before a specified time point. Table 4.1 shows the functions from the C++ Standard Library that can accept timeouts, their parameters, and their return values. Parameters listed as *`duration`* must be an instance of `std::duration<>`, and those listed as *time_point* must be an instance of `std::time_point<>`.

Table 4.1. Functions that accept timeouts

Class/Namespace	Functions	Return values
std::this_thread namespace	sleep_for(*duration*)sleep_until(*time_point*)	N/A
std::condition_variable or std::condition_variable_any	wait_for(*lock*,*duration*)wait_until(*lock*,*time_point*)	std::cv_status::timeout or std::cv_status::no_timeout
	wait_for(*lock*,*duration*,*predicate*)wait_until(*lock*,*time_point*,*predicate*)	bool—the return value of the *predicate* when awakened
std::timed_mutex or std::recursive_timed_mutex	try_lock_for(*duration*)try_lock_until(*time_point*)	bool—true if the lock was acquired, false otherwise